Ali Ezzat, PhD

Principal Data Scientist — Deep Learning, Real-Time ML Systems, Optimisation & Recommender Systems

aliezzat1985@gmail.com · linkedin.com/in/aliezzat1985 · github.com/alizat

PROJECT PORTFOLIO

Twenty-seven machine learning and analytics engagements across financial services, industrial manufacturing, supply chain, retail and loyalty, healthcare, education and mobility. Each is presented as a STAR case study — the situation and task that defined it, the solution I built, and the outcome it delivered. Entries marked (PoC) were proof-of-concept or pilot engagements.

Contents

CREDIT RISK & LENDING

GENERATIVE AI & LLM SYSTEMS

INDUSTRIAL & SUPPLY CHAIN OPTIMISATION

FORECASTING, PRICING & CUSTOMER ANALYTICS

#	Project	Headline result
14	Western Union (IBAG) - Branch Cash-Withdrawal Forecasting	Over-supply error 300% → 20%
15	Wayak Health - Patient Lead Scoring, Retention and KPI Analytics	Targeting precision 3% → 25%
16	Dsquares - Loyalty Merchant Recommender and RFM Segmentation (PoC)	Cross-merchant recommender over three loyalty programmes
17	AMAN Financial Services - Merchant Churn Prediction	79% precision on 100GB+ POS data
18	London Cab - Driver Positioning and Pricing Simulation	13% faster passenger pickup
19	Naqla - Truck-Trip Price Analytics and Estimation	Automated trip-price estimation + driver analytics
20	Sa3ar - Used-Car Fair-Price Estimation	Market dataset scraped and priced from scratch
21	AUC - Course Registration Demand Forecasting (PoC)	Per-course seat forecasts from a course-dependency graph
22	Toyota - Customer Data Quality and Deduplication	Multi-identifier entity resolution
23	Cargolinx - Commodity Name Standardisation (PoC)	Fuzzy-matched commodity taxonomy for shipment data
24	Abou Ghaly Motors - Service-Visit Analytics (PoC)	Odometer-rate outlier detection across 1,100+ vehicles

RESEARCH & OPEN SOURCE

#	Project	Headline result
25	dbparser - Open-Source Drug-Database Integration Package	50K+ downloads, active community adoption
26	PhD Research - Drug-Target Interaction Prediction	4 papers + 1 book chapter, InCoB 2016 Best Paper

EARLIER ENGINEERING

#	Project	Headline result
27	Earlier Software Projects - BRS and OTMS	3 years enterprise Java delivery, on-site deployment

CREDIT RISK & LENDING

Credit Risk ML Platform - Multiple Financial Institutions

CONTEXT

Situation. A dozen financial institutions across Egypt were making lending decisions without reliable, scalable credit assessment. Each one carried default rates it could not explain and could not reduce, and each was reviewing applications manually or through rules that no one had validated against outcomes. They were struggling to increase the sizes of their portfolios while keeping the default rates down.

Task. Architect and deploy credit-scoring pipelines for all such clients. Each pipeline had to do more than predict risk: it had to return an accept/reject decision *and* a defensible loan amount, using the client's own income-estimation formulas, debt-burden rules and lending limits.

Challenge. A model alone was never the deliverable. Credit-bureau data, eligibility criteria, policy rules and loan-limit arithmetic all had to be operationalised consistently and exposed to the client's systems as something their engineers could call in production. Every client had different policy, different data, and different tolerance for risk.

SOLUTION

Action. I built GOOD/BAD credit-scoring engines paired with client-specific policy engines. The models ingest features parsed from applicant credit-score reports and return an accept/reject decision alongside a recommended loan amount. I owned the entire lifecycle — data wrangling, EDA, feature engineering, model training, Docker containerisation, API development, deployment, production support, and direct intervention when incidents hit production.

In most cases, the system includes an iScore report parser, an association-analysis workflow that surfaces feature-value patterns correlated with good and bad credit behaviour, trained model and PCA artifacts, a simulation harness, and maintained policy lists covering geographic areas, professions and industry sectors. I shipped it as a containerised API with an OpenAPI specification, sample request/response payloads, local and deployed simulation scripts.

The policy layer is explicit and versioned rather than buried in code. Alongside the model artifact and its PCA transformation, each deployment carries editable reference lists — restricted and permitted governorates and cities, positive and negative area tiers, restricted professions and industry sectors, iScore loan types of interest, an iScore risk categorisation table, and the flags governing when income may be assumed rather than documented. Credit-bureau reports arrive as raw strings; a companion parser package turns them into structured repayment and inquiry histories, aggregates them into features, and parallelises the parse across ten cores so a month of reports can be processed in one pass. The same simulation script drives either the locally built container or the model deployed on Konan, so a release can be validated identically before and after it ships.

Rationale. Predictive signal on its own is not an operational decision. Fusing the model output with explicit, client-owned credit policy is what makes the answer usable by a loan officer and auditable by a risk committee. The simulation and API test layers exist so that every release can be validated against realistic traffic before it touches a live applicant.

OUTCOMES

Result. Across the twelve deployments, the platform processes **100K+ loan applications per month** and has facilitated **$150M+ in lending**. Average default rates fell **40%** against clients' pre-engagement baselines.

I also served as applied ML lead for Konan, Synapse Analytics' internal MLOps platform, translating what I learned running these pipelines into platform requirements — Konan now supports roughly 25 production models across about 15 client deployments.

RayaTrade - Collections Prioritisation Model

CONTEXT

Situation. RayaTrade needed to chase roughly **45,000 borrowers a month** about their upcoming instalment. Calling all of them was not viable — the collections team had a fixed number of agents and a fixed number of hours, and was spending them uniformly across customers who were overwhelmingly going to pay on time anyway.

Task. At the start of each month, tell operations exactly which funded customers to call, ranked by how much the call was likely to matter.

Challenge. The relevant failure is an instalment delayed by more than 30 days. Predicting that is only half the problem; the output had to arrive as something a collections supervisor could act on Monday morning without interpretation.

SOLUTION

Action. I built a gradient-boosted (XGBoost) model trained on instalment-payment history, with features drawn primarily from each customer's behaviour on preceding instalments. It predicts good/bad behaviour on the current instalment and emits a call list ordered by the likelihood that the customer will miss the payment date. The pipeline covers dataset construction, feature utilities and monthly call-list generation end to end.

Because the deliverable is a ranked list consumed under a fixed agent capacity, I evaluated it as a ranking problem rather than a classifier. The evaluation utilities compute per-class precision, recall and F-score at any decision threshold, and — more importantly — **precision and recall at top-k and at bottom-k of the ranked list**: how much of the actual bad behaviour is captured in the first *k* calls the team can afford to make, and how clean the tail is that they skip. Sweeping *k* across plausible team capacities turns model quality into an operational answer — how many calls buy how much of the risk.

Rationale. Risk ranking converts a scarce resource — human contact time — into a prioritised queue, and the handoff is deliberately simple: one ordered list per month, no dashboard to interpret, no model output for a non-technical user to second-guess. Evaluating at top-k rather than at a global threshold is what makes the metric mean something to the collections manager, since their constraint is agent-hours, not a probability cut-off.

OUTCOMES

Result. The model turns an undifferentiated 45,000-borrower calling problem into a ranked worklist, allowing collections effort to concentrate on the customers most likely to fall past 30 days, with capacity-aware top-k evaluation showing how much delinquency each additional block of calls recovers.

Tamweely - Credit Scoring Model and Deployment API

CONTEXT

Situation. Tamweely needed credit scoring built entirely from its own database — no parsed iScore reports, no external bureau features to lean on. Development ran on the Tamweely server, with the work backed up to my repository.

Task. Predict GOOD or BAD repayment behaviour from a customer profile, where BAD means any payment delayed by more than 15 days. Deploy it as an API and validate it against live production data.

Challenge. A 2025 database migration changed the operating environment underneath the model. Validation was therefore not a formality: it required agreement with the client on label definitions, verification of request-schema integrity, sanity checks on response distributions, and enough matured applications to actually evaluate outcomes.

SOLUTION

Action. I built the full R workflow — data wrangling, feature engineering, EDA, model training and deployment testing — and packaged it as a Dockerised deployment bundle with an API server, documented endpoints, an OpenAPI specification, the trained model artifact, local and deployed test suites, and a documented Konan release process.

I also wrote the operating guide the client runs against production: checks for missing and wrongly typed fields, detection of suspiciously constant fields, monitoring of GOOD/BAD response ratios, and reconciliation against a client-supplied labelled test set.

Rationale. Most credit models fail in production not because the model is wrong but because the data arriving at the endpoint quietly stops matching what the model was trained on. Explicit, written validation procedures catch schema and distribution regressions early — and settling the definition of "bad repayment" in advance prevents the disagreement that otherwise surfaces months later when the numbers are disputed.

OUTCOMES

Result. Delivered a deployed, documented scoring API with an operating guide the client can run independently. Post-deployment analysis established that 34% of accepted applications in 2025 were BAD — a baseline that quantifies the size of the problem the model exists to address.

B.TECH - Credit-Scoring Modelling and Production Scorer

CONTEXT

Situation. B.TECH extends consumer financing at retail point of sale, where a credit decision has to be made in the time a customer is standing at a counter — and where the information available at that moment varies.

Task. Train, test, simulate and package credit-risk models for operational scoring.

Challenge. One model could not serve every decision point. The system needed distinct accept/reject and good/bad models, each with variants that use or exclude iScore bureau information, reflecting the reality that bureau data is not always available at the moment of decision.

SOLUTION

Action. I implemented an R pipeline covering cleaning, labelling, model development, targeted testing, production simulation, production training and scorer preparation. The delivered artifacts include trained RDS models for both accept/reject and good/bad variants, a production scorer, a test suite and Docker build configuration.

Rationale. Keeping training, testing, simulation and scoring as separate, versioned artifacts is what makes a credit model reproducible a year later — when a regulator, a client, or a successor asks why a specific applicant was rejected.

OUTCOMES

Result. A production-packaged scorer covering multiple decision stages and information-availability scenarios, within a credit-scoring portfolio averaging **74% precision** on credit-worthiness prediction.

Additional Credit-Scoring Engagements - EGBank, Zeyada and Sarwa

CONTEXT

Situation. Alongside Tamweely and B.TECH, I delivered credit-scoring work for EGBank, Zeyada and Sarwa — three further financial institutions within the twelve-client credit-risk programme, each with its own data, policy and risk appetite.

Task. Develop credit-worthiness assessment and loan-amount decisioning models, and maintain, experiment with and test the credit-risk policy workflows around them.

Challenge. Each client's lending policy differed, so the modelling approach had to transfer while the policy layer was rebuilt per institution.

SOLUTION

Action. I deployed ML models that assess an applicant's credit-worthiness and determine the loan amount to grant, supported by association analysis identifying sets of feature-value pairs that correlate with good and bad credit behaviour, and facilitated the maintenance, experimentation and testing of each client's credit-risk policy workflows.

Rationale. Association analysis complements the model by producing something a credit committee can read: explicit, human-legible patterns tied to good and bad outcomes, which supports policy discussion in a way a model score alone does not.

OUTCOMES

Result. Three further institutions brought onto the credit-scoring platform, within a portfolio averaging **74% precision** on credit-worthiness prediction.

GENERATIVE AI & LLM SYSTEMS

Raya - Alternative Credit Signals from SMS

CONTEXT

Situation. Thin-file applicants — people with little or no credit-bureau history — are effectively invisible to conventional credit scoring. They are also a large share of the addressable market. Their phones, however, hold a detailed financial record: transaction alerts, salary notifications, bill reminders, card activity.

Task. Build an embeddable service that converts raw SMS inboxes into structured financial indicators — transaction activity, income and expense signals, banking relationships, cards, merchants and spending categories — usable as credit-scoring features.

Challenge. Banks, mobile operators, lenders, apps and subscription services all format their messages differently, in more than one language. The system had to identify transactions, amounts, currencies, senders and entities across that heterogeneity while confidently discarding the noise — OTPs, marketing, service notices.

SOLUTION

Action. I built a multilingual NLP pipeline on GPT-4o that extracts income, spending behaviour and vendor-category signals from unstructured SMS for thin-file alternative credit scoring. The service is a Python API with transaction filters, entity retrieval, currency processing, aggregators, Docker packaging and unit tests.

The core design masks transaction bodies and matches them against a library of reusable templates. Matched messages yield structured entities immediately; unmatched messages are captured for periodic review so the template library grows as message formats change.

Model benchmarking and cost reduction. I evaluated several frontier LLMs on the extraction task itself — message bodies mixing English and Arabic — scoring each against a hand-labelled ground-truth set, so the choice of model was measured rather than assumed. That hand-labelled set then became the training data for the smaller models. Frontier-model API calls are not economical at per-applicant credit-scoring volume, so I ran two approaches at the cost problem. The first was distillation: a QLoRA fine-tune of Llama 3.1 8B, using Unsloth on Colab and trained on synthetic data generated by GPT-4o, to test how much frontier capability transferred into a small open model. The second, and the approach the pipeline ended up using, was a fine-tune of AraEuroBert — a multilingual model from HuggingFace — on the hand-labelled set. Not one model doing everything: a first model classifies the message type, bank, promotion, network operator and so on, and separate fine-tuned models parse the individual features from there, one per feature. Splitting the problem this way keeps each model small, each label set narrow, and each failure attributable to a specific extractor.

Orchestration, and why it was not adopted. A colleague on this project built a LangGraph state machine as an attempt to improve extraction quality — a classifier node typing each message, conditional routing to typed extraction nodes, schema-validated output. It produced some interesting results, but it was never used: it spent too many tokens on frontier-model API calls to be affordable at production volume. Underneath the model layer, deterministic primitives — amount extraction, currency handling, date parsing, transaction detection, entity recognition, network-operator handling — each carry their own unit-test suite with expected outputs, so regressions surface at the primitive rather than in an end-to-end diff.

Rationale. Template-backed extraction makes recurring transactional messages cheap, traceable and debuggable, and reserves model inference for the cases that actually need it. The unmatched-message queue is what keeps the system from silently degrading as banks change their SMS formats. Typing the message first and then parsing features with dedicated small models — rather than asking one large model to do everything — keeps each label set narrow and each error attributable, and it is what makes per-applicant processing economical at the volumes credit scoring requires. The agent experiment is the counter-example that proves the point: it worked, and it still lost to cost.

OUTCOMES

Result. The pipeline gives credit decisioning a structured financial profile for applicants who would otherwise be unscoreable, at an inference cost low enough to run per application — arrived at by measuring frontier models against a hand-labelled set and then replacing them with small fine-tuned encoders, rather than by assumption.

Forsa - LLM Customer Activation Assistant

CONTEXT

Situation. Forsa could not staff the top of its funnel. Prospective customers who had filled in an expression-of-interest form still had to be nudged through to a completed limit application, and every one of those conversations was repetitive, high-volume and sensitive to tone.

Task. Build an assistant that holds a real conversation with a prospect — drawing on their stated interests and the offers currently available from affiliated merchants, answering questions about the app and its services, and steering toward a completed application.

Challenge. This is regulated financial services conducted in Egyptian Arabic. The assistant had to stay grounded in company policy, sound natural rather than machine-translated, and never invent an offer or a term.

SOLUTION

Action. I built a conversion assistant on GPT-4o that loads customer interests and offer data, selects the relevant offers for the person it is speaking to, and injects that customer context together with background company knowledge through the system prompt to steer Egyptian-Arabic conversations toward activation. It runs behind a Streamlit interface for demonstration and internal testing, with the background knowledge kept as an editable document rather than embedded in code.

Rationale. For a bounded, well-documented policy and offer surface, prompt-grounded context delivers compliance and personalisation without the operational cost of a vector store and retrieval tier — and keeping the background document separate from the prompt code means the business can correct what the assistant knows without a release.

OUTCOMES

Result. An assistant that turns a static expression-of-interest list into individual, offer-aware conversations in the customer's own dialect, covering the activation end of the customer lifecycle.

Forsa - WhatsApp Collections Agent

CONTEXT

Situation. Chasing overdue instalments is the other conversation Forsa could not staff. Every late customer needs contacting repeatedly, in their own dialect, with a tone that has to change as the delay lengthens — friendly at day three, firmer at day twenty — and with an outcome that has to be recorded, not just discussed.

Task. Build an agent that contacts overdue borrowers over WhatsApp, negotiates a payment date or a partial-payment plan, keeps the resulting commitments as state the business can act on, and knows when to stop and hand over to a human.

Challenge. A collections conversation is a policy surface, not a chat. The agent had to handle customers who promise a date and then move it, who ask to pay in instalments, who ask what iScore is, who reply in Franco-Arabic, who are confused about why they are being contacted at all — and it had to do this inside Twilio's 24-hour session rules, where a conversation that has gone quiet can only be reopened with a pre-approved template.

SOLUTION

Action. I built a Twilio WhatsApp agent driven by an explicit operating policy. It retrieves same-day conversation history, sends template-based conversation starters when no session is open, and generates JSON-structured replies through an OpenAI model so every turn returns something the application can validate and act on.

The behaviour is tiered by how late the customer is: days one to six, helpful and friendly with no partial payments offered; days seven to fourteen, still friendly but partial payments allowed; days fifteen to twenty-nine, a firmer tone; past thirty days, the agent stops and the case goes to people. Around that sit the stateful parts — extracting the payment date a customer states and updating it when they revise it, tracking first and second partial payments separately, and writing any concern the customer expresses back to the database against their application. The agent understands Arabic, English and Franco-Arabic and replies in formal English or Egyptian colloquial Arabic. A hard guardrail ends the exchange after ten model turns, so no conversation can run away.

Rationale. The tone tiers and the thirty-day cut-off are the design: they encode the client's collections policy into the agent rather than trusting a model to be appropriately firm, and they guarantee that the hardest cases reach a human instead of a bot. Structured JSON output is what lets a promise made in conversation become a due-date change in the database. And the turn limit is the cheapest possible safety property — whatever else goes wrong, the conversation ends.

OUTCOMES

Result. A collections agent that conducts the full late-payment conversation — contact, negotiation, partial-payment arrangement, commitment capture and escalation — over the channel customers actually read, replacing an agent-per-contact model for the earlier stages of delinquency while reserving human effort for the cases past thirty days.

INDUSTRIAL & SUPPLY CHAIN OPTIMISATION

Unilever - Real-Time Detergent Manufacturing Recommender

CONTEXT

Situation. A detergent manufacturing tower burns natural gas continuously to dry slurry into powder. Gas is one of the largest controllable costs in the process — but every lever that reduces consumption also risks pushing moisture content or bulk density outside specification, which scraps the batch.

Task. Deliver a decision-support system that runs continuously on live sensor data and recommends machine control settings that cut energy use without compromising product quality.

Challenge. Operators cannot be handed recommendations they must simply trust. The system had to reason over changing slurry flow, nozzle configuration, burner controls, fan speed, tower pressure, multiple temperature readings and recipe-specific constraints — and it had to fail safely, because a quality excursion costs more than the gas it saves.

SOLUTION

Action. I designed and deployed a real-time recommendation system operating on live data from the tower, drawing on **more than twenty tagged inputs** — temperature at nine points across the tower, moisture content, bulk density, natural-gas flow and real-time gas consumption, tower pressure, slurry flow, slurry bypass and return states, burner on/off state, high-pump speed, burner- and pressure-fan speeds, nozzle state and recipe ID. It periodically ingests that machine state and recommends four control settings back: nozzle openings, high-pump speed, natural-gas potentiometer setting and burner-fan speed.

The engine uses min-max-normalised nearest-neighbour retrieval over historical operating logs: for the current conditions, it finds past machine states that ran under matching conditions with a favourable energy profile, and recommends those settings. Around it sits the quality logic. Moisture is not treated as in-spec or out-of-spec but banded into five zones — very low, low, optimal, high, very high — whose boundaries come from the **recipe's own gauge thresholds** rather than a global constant, so the same code enforces a different envelope for every product. The slurry-to-nozzle ratio is carried as a derived control feature, and a nozzle-count-to-pump-speed target table gives the system a known-good fallback configuration. When moisture or bulk density leaves the target band, control passes from the ML recommendation to rule-based corrective actions — adjusting flame strength, opening additional nozzles or increasing slurry supply — and returns to ML-driven recommendations once the process is back in range.

I shipped it as a containerised REST service (plumber API behind Docker) with unit tests covering both the recommendation logic and the API layer, file-based logging of every recommendation, and sample request payloads for the awkward cases specifically — moisture far above target, no slurry entering the tower, a process that is not yet steady — so the edge behaviour is testable and not just documented.

Rationale. Nearest-neighbour retrieval was chosen deliberately over a black-box regressor: every recommendation can be traced back to a real, historical run of the same tower under the same recipe, which is what earns an engineer's trust on a live production line. Driving the moisture bands from recipe parameters rather than hard-coding them is what let the system generalise across products without a code change per variant. The rule-based fallback is what makes the whole thing safe to deploy — it guarantees that the worst case is a return to conventional operation, not a ruined batch — and the edge-case payloads are what proved that guarantee before it was tested by a real one.

OUTCOMES

Result. 23% reduction in monthly natural-gas consumption, with **zero production batch failures** after the quality safeguards were introduced. The system ran continuously on live manufacturing data as a production decision-support tool.

Unilever - Multi-Variant Manufacturing Scheduler

CONTEXT

Situation. Beyond the tower itself, Unilever had to decide the order and allocation of production for multiple detergent variants against a monthly demand sheet. Planners were building those schedules by hand, reconciling constraints in their heads.

Task. Build a tool that generates production schedules minimising total manufacturing time for a given demand scenario.

Challenge. A valid schedule has to respect line speeds, line efficiencies, variant specifications and base-powder inclusion levels simultaneously. Changing one demand figure ripples through all of them, which is exactly what makes manual scheduling both slow and hard to optimise.

SOLUTION

Action. I built an R Shiny application with reusable scheduling functions, demand-sheet templates, scenario inputs and lookup tables encoding variant specifications, line speeds, line efficiencies and inclusion levels.

Rationale. Parameterising the constraints rather than hard-coding a single schedule turns planning into an experiment: a planner can test a demand scenario against the real manufacturing envelope in seconds, instead of constructing one feasible schedule by hand and hoping it is close to optimal.

OUTCOMES

Result. Planners gained a scenario-testing tool that generates time-minimising schedules directly from a demand sheet while respecting the full constraint set of the manufacturing pipeline.

Procter & Gamble - Supply Network Design Optimisation

CONTEXT

Situation. P&G's distribution network to retail stores had grown incrementally rather than by design — warehouses, hubs, routes and fleet sized by history rather than by optimisation. Nobody could say how much that legacy configuration was costing.

Task. Recommend warehouse, branch and hub locations, delivery routes, fleet requirements and shipment frequencies that minimise total supply-network cost — while supporting brown-field scenarios where existing facilities must stay put.

Challenge. This is four coupled problems at once: geographic allocation, capacity planning, facility location and vehicle routing, across a **multi-echelon network** — megahubs feeding hubs, hubs feeding branches, branches feeding stores — where a decision at one tier changes the cost of every tier below it. Solving them jointly at network scale is intractable, and the answer still has to be something a logistics manager can look at on a map and believe.

SOLUTION

Action. I built a full supply-network-design optimisation platform. Facility locations are found by clustering demand geographically, with centres initialised by a **farthest-point (max-min) rule rather than at random**, so a run does not converge onto a degenerate configuration and results are reproducible across runs. Stores are assigned to branches and branches to hubs, decomposing the network into serviceable areas, and a travelling-salesman routine then builds **milk runs at two levels** — branch-to-store and hub-to-branch — within each area.

The cost model is where the realism sits. Trucks have capacities and rental rates, and operating cost is derived from rent by an explicit factor. Warehouse footprint is a function of the throughput it serves, so facility cost scales with what a site actually handles rather than being a flat per-site number. Routing is bounded by operational reality: a truck working day of nine hours on branch-to-store runs and twelve on hub-to-branch, a per-store unload time that grows with drop size, a per-branch stopover time, a maximum branch-to-store distance, and a service-probability multiplier reflecting that not every store is visited in every cycle. The platform then **sweeps the whole configuration space** — candidate branch counts against candidate hub counts, on the order of 35–50 branches by 2–8 hubs — and reports the cost of every combination rather than a single answer.

I integrated the HERE Maps API so routing used real road distances rather than straight lines, and built interactive geographic visualisations with Leaflet and an R Markdown interface. A brown-field mode lets selected existing facilities be pinned while the rest of the network is optimised around them, and the whole tool is driven by CSV templates — stores, existing branches, megahubs, truck types, supply routes — so it re-runs against a different network without touching the code.

Rationale. Clustering makes the problem tractable by decomposing it into independent service areas; routing then optimises transport within each. Deterministic centre initialisation matters more than it sounds: a client challenging a recommendation needs the same inputs to produce the same map. The cost-comparison sweep matters as much as the optimum itself — it shows the client the shape of the cost curve, so they can see what a two-hub network costs versus three and make the trade-off themselves. Brown-field mode is what makes any of it actionable: a recommendation that requires closing every existing facility is a recommendation that never gets implemented.

OUTCOMES

Result. 18% reduction in total supply-network cost versus the client's existing configuration, delivered with interactive geographic visualisations that let operational stakeholders inspect and challenge each recommendation, and a cost surface across every hub/branch combination rather than a single point recommendation.

Grinta - Medicine Inventory Replenishment Forecasting

CONTEXT

Situation. Grinta supplies medicines to pharmacies and was losing money at both ends of the same decision: unsold stock accumulating carrying fees, and out-of-stock products turning into missed sales. Both are symptoms of ordering without a forecast.

Task. Forecast replenishment requirements so stock levels are set before supplying pharmacies, not corrected afterwards.

Challenge. The two error directions carry different costs, and pharmaceutical inventory adds shelf-life constraints to an already asymmetric problem — over-ordering is not simply capital tied up, it can be product written off.

SOLUTION

Action. I built a time-series forecasting model for medicine inventory replenishment, predicting how much to stock ahead of supplying pharmacies.

Rationale. Replenishment is a recurring, per-product decision, which makes it exactly the kind of judgement that benefits from being made by a forecast rather than by a person applying a rule of thumb across a long catalogue.

OUTCOMES

Result. Reduced inventory fees on unsold product and avoided missed sales on out-of-stock items — improvement on both sides of the trade-off simultaneously.

Pharmaceutical Manufacturer - Field-Force Visit and Travel-Cost Analytics (PoC)

CONTEXT

Situation. A pharmaceutical manufacturer's medical representatives visit doctors and pharmacies across the country, log those visits in a CRM, and claim travel expenses against them. The two records were never reconciled. Nobody could see what a representative's day actually looked like on a map, or whether the distance implied by their visits matched the distance they were reimbursed for.

Task. Turn the CRM visit log and the expense claims into a view of field activity that supervisors can interrogate — per representative, per day and in aggregate.

Challenge. Visits arrive as GPS coordinates attached to account names, expenses arrive per representative per day in a separate reporting system, and territories are defined in yet another vocabulary (IMS bricks) with account names that do not match cleanly across sources. Any reconciliation had to survive that mismatch before it could say anything about a claim.

SOLUTION

Action. I built an R Shiny application, access-controlled per user, that reconstructs each representative's day as a route on an interactive map: every logged visit plotted from its coordinates and annotated with timestamp and account name, connected in visit order so the shape of the day is visible at a glance. A second view collapses a representative's entire period into the set of accounts they cover, showing territory footprint rather than a single day. Behind it I assembled the supporting reference data — an inter-governorate distance matrix, per-representative-per-day travel expense records, monthly expense summaries and an account-name matching table — so claimed cost could be read against the geography actually covered.

Rationale. Field-force spend is argued about anecdotally because nobody can picture the day being paid for. Drawing the route first, and only then putting the expense figure beside it, moves the conversation from a spreadsheet dispute to something a line manager and a representative can look at together.

OUTCOMES

Result. A proof of concept giving field-force supervisors per-representative daily route reconstruction, territory coverage views and the reference data needed to read travel claims against the distances they imply.

FORECASTING, PRICING & CUSTOMER ANALYTICS

Western Union (IBAG) - Branch Cash-Withdrawal Forecasting

CONTEXT

Situation. Western Union branches were being supplied with cash on estimates that were wrong by a wide margin — over-supply error was running at **300%**. Idle cash in a branch is capital that earns nothing and carries security and insurance costs; under-supply turns a customer away at the counter.

Task. Forecast daily cash-withdrawal amounts at individual branch level.

Challenge. Branch-level demand is far noisier than aggregate demand, and the two failure modes are asymmetric — costly on one side, reputationally damaging on the other. The forecast had to be granular enough to drive per-branch supply decisions.

SOLUTION

Action. I developed a time-series forecasting model predicting daily money-withdrawal amounts for each branch.

Rationale. Forecasting per branch rather than centrally is what makes the output actionable, since cash is physically delivered branch by branch.

OUTCOMES

Result. Cash over-supply error reduced from 300% to 20% — a fifteenfold improvement in supply accuracy.

Wayak Health - Patient Lead Scoring, Retention and KPI Analytics

CONTEXT

Situation. Wayak was launching an online drug-delivery subscription and calling patients essentially at random. Roughly 3 in 100 converted, which made outreach economics untenable. The same blindness applied at the other end of the lifecycle: nobody knew which existing customers were about to stop ordering.

Task. Deliver the exploratory analysis and KPI visibility the business needed to get off the ground, a lead-scoring workflow that ranks patients by conversion likelihood, and a retention model identifying customers worth calling back.

Challenge. The raw material — patient records, call logs, orders, batch data, chronic-medication histories — needed substantial preprocessing and feature engineering before any repeatable outreach list could be produced, and the output had to fit the sales team's existing weekly rhythm. Patient identity itself was unreliable: names and phone numbers arrived in inconsistent formats across the CRM, the call centre and the order system.

SOLUTION

Action. I ran the extensive exploratory analyses the client requested to orient the business, and built a dashboard surfacing their KPIs of interest, including order-aging views. On top of that I built the lead-scoring pipeline end to end: data extraction, preprocessing, feature engineering, model training, batch sorting and database injection — with weekly processing of call activity, order history, patient behaviour and disease-related features.

The data layer. Rather than one preprocessing script, the pipeline is a **layered SQL view stack**: reusable SQL functions for name and phone normalisation at the bottom, then cleaning views (unique customers, cleaned calls, formatted patients), then feature views (per-customer order aggregates, incomplete-order flags, chronic-drug reorder timing, call-to-order joins), then the serving views the models and the weekly batches read from. A script runner materialises the layers in order. This is what makes identity resolution and feature definitions shared rather than re-implemented per model, and it is why a weekly batch can be regenerated reproducibly.

The models. Conversion scoring is XGBoost, tuned over a hyperparameter grid and evaluated on precision, recall, F-score and feature importance — with parallel zone-aware and zone-free variants, since geographic coverage was not uniform and a model leaning on zone would have been unusable in areas without delivery. Separately I built a **retention model** (an XGBoost classifier predicting whether a customer returns, wrapped as a reusable class with its own preprocessing, encoding and metric reporting). The two models are then combined: a patient's score is the higher of their new-sales and retention probabilities, and the ranked batch is cut at a probability threshold before it goes to the call centre — so the sales team receives one list, whichever reason the patient is worth calling for.

Rationale. The model's output is written back to the database as sorted batches rather than surfaced as scores in a dashboard, because the sales team's workflow is a call list, not an analysis. Making the prediction arrive in the shape the operator already works in is what got it used. Building the feature layer as versioned SQL views rather than in-model code is what allowed a second model to be added without rebuilding the first one's inputs. And combining acquisition and retention scores into a single ranked list respects the fact that the constraint is call capacity, not model count.

OUTCOMES

Result. Targeting precision improved from 3% to 25% — roughly an eightfold improvement in the conversion rate of outreach — delivered as a weekly ranked batch spanning both new-conversion and win-back candidates, on top of a reusable SQL feature layer and a KPI dashboard the business ran on.

Dsquares - Loyalty Merchant Recommender and RFM Segmentation (PoC)

CONTEXT

Situation. Dsquares runs loyalty and rewards programmes on behalf of large brands — among them two major banks and a mobile operator. Across those programmes it holds the redemption history of a very large customer base at hundreds of retail merchants, and was using it for reporting rather than for targeting. Nobody could say which merchants were complements, which customers were slipping away, or what to put in front of a given member next.

Task. Turn multi-programme redemption history into three things the commercial team could act on: a map of how merchants relate to one another, a segmentation of customers by value and recency, and a per-customer next-merchant recommendation.

Challenge. The same retailer appeared under many spellings across the three programmes — punctuation, accented characters, abbreviations, trailing branch names — so any co-occurrence analysis would have split one merchant into several before it started. Merchant categories were inconsistent across programmes too. The identity problem had to be solved before the interesting work could begin.

SOLUTION

Action. I started with **name unification**, using a purpose-built string distance rather than an off-the-shelf one: it compares names character by character from the start and weights each position by an exponential decay, so the measure is dominated by whether two names begin the same way — which is how retail brand names actually differ from one another. That produced a merchant transformation table mapping every observed spelling onto a canonical merchant, plus a reconciliation of category labels across the three programmes.

On the unified data I built each customer's redemption basket per programme and ran **association-rule mining** over those baskets, retaining rules by support, confidence and lift. The recommender selects, for each merchant a customer already uses, the rule whose **confidence jump** over the base rate is largest — the merchant whose likelihood is most changed by what the customer already does, rather than simply the most popular one. Rules were mined at both merchant and category level.

I also built a **merchant-proximity map**: co-occurrence (Jaccard) and confidence matrices between merchants embedded into two dimensions with t-SNE and plotted interactively, coloured by retail category and filterable by how frequently a merchant appears — so a category manager can see which merchants cluster together and which sit apart from their nominal category. Alongside it I built an **RFM segmentation** that computes recency, frequency and monetary segments over multiple time grains and compares each period against the one before it, so segment migration is visible rather than just the current snapshot. All of it was delivered as an interactive Shiny dashboard, including a rule explorer for the commercial team to interrogate the association rules directly.

Rationale. Popularity-based recommendation in a loyalty programme mostly recommends what the customer would have found anyway; ranking by confidence jump is what makes a suggestion informative rather than obvious. The t-SNE map exists because a category manager will not read a rules table but will read a picture — and disagreements between the picture and the official category taxonomy were themselves a finding. Period-over-period RFM was chosen over a static segmentation because the actionable question is who is moving *out* of a valuable segment, not who is currently in one.

OUTCOMES

Result. A proof of concept covering the full chain from raw multi-programme redemption data to commercial action: canonical merchant identities, a mined rule base at merchant and category level, a next-merchant recommender ranked by confidence lift, a visual merchant-proximity map, and RFM segments tracked across periods — all explorable by non-technical users in one dashboard.

AMAN Financial Services - Merchant Churn Prediction

CONTEXT

Situation. AMAN held **100GB+ of point-of-sale transaction data covering 100K+ merchants** and had almost no visibility into why merchants stopped transacting. Churn was being discovered after the fact, when the revenue had already gone.

Task. Make the data tractable, deliver insight to stakeholders, and predict which merchants were about to churn.

Challenge. The volume and the transaction-level granularity made preparation the hard part — the modelling could not start until 100GB of raw POS records had been wrangled into merchant-level behavioural features, and the findings had to be communicated to non-technical stakeholders.

SOLUTION

Action. I wrangled the full 100GB+ POS dataset, mined it for insight and delivered those findings to stakeholders through visualisations and reports, then developed an XGBoost model to predict merchant churn.

Rationale. Gradient boosting suits tabular behavioural features derived from transaction aggregates, and gives feature-importance output that made the churn drivers legible to the business — which was half the value of the engagement.

OUTCOMES

Result. Merchant churn prediction at **79% precision**, plus a stakeholder-facing view of the behavioural patterns preceding churn.

London Cab - Driver Positioning and Pricing Simulation

CONTEXT

Situation. Passengers were waiting too long for pickup, and drivers were waiting in the wrong places — supply sitting idle in one part of the city while demand went unserved in another. Separately, the business wanted to change pricing but had no safe way to test what a change would do.

Task. Identify better driver waiting locations, and give the business a way to experiment with pricing strategies before deploying them.

Challenge. Driver supply must be positioned *before* ride requests arrive, which makes this a forecasting problem disguised as a geography problem — and demand hotspots move by hour of day and day of week. Pricing changes, meanwhile, affect both sides of a marketplace at once, so live experimentation is expensive to get wrong.

SOLUTION

Action. I began by reverse-engineering the operational database itself — enumerating every table in the production SQL Server instance, profiling row and column counts and sampling each one, because no documentation existed describing where trips, tickets, drivers and areas actually lived. From the ticket history I reconstructed the quantities that were not recorded directly: the response distance between an available driver and a ride request, and driver availability at the moment each ticket was created.

On that base I built temporal clustering to identify optimal driver waiting locations — internally, the "magnetizer". It is a **custom k-means run on great-circle distances rather than Euclidean ones**: cluster membership is assigned by geodesic distance and centres are updated and re-assigned until membership stops changing, so a "centre" is a point that is genuinely closest by geography rather than by planar coordinates. Demand is sliced by **time of day** (late night, morning, afternoon, evening) and day of week, so each slice gets its own set of waiting points. Results are presented on an interactive Leaflet map inside a dashboard. I also designed a dashboard for experimenting with pricing strategies and observing the outcomes through simulation.

Rationale. Euclidean k-means on latitude and longitude quietly distorts distance, and in a positioning problem the whole output *is* a distance claim — so the clustering had to run on the real metric. Clustering on time as well as space is the other half: a location that is optimal at 8am is not optimal at 11pm, and a static hotspot map would have positioned drivers correctly only part of the day. Simulation lets the business explore pricing outcomes without risking a live marketplace.

OUTCOMES

Result. Passenger pickup times reduced by 13%, plus a simulation environment for testing pricing strategy before deployment, built on a reconstructed view of an undocumented production database.

Naqla - Truck-Trip Price Analytics and Estimation

CONTEXT

Situation. Naqla's truck-trip prices were set by negotiation and experience rather than by any explicit model, which made quoting inconsistent and left the business unable to explain what actually drove a price.

Task. Analyse trip data to expose the real drivers of price, and build a model that estimates the price of a shipment from its characteristics.

Challenge. Price depends on a wide, interacting set of factors — truck type, shipment weight and goods type, distance travelled, origin and destination, toll fees — none of which is individually decisive.

SOLUTION

Action. I built an analytical dashboard for investigating the patterns governing trip prices across these dimensions, then developed both deep-learning and XGBoost regression models to estimate prices for trips made by trucks of varying size carrying different goods types.

Rationale. The dashboard and the model answer different questions and both were needed: the dashboard tells the commercial team *why* prices vary, which is what they can act on in negotiation, while the model gives a number for a specific shipment. Building both a neural and a gradient-boosted regressor allowed a direct comparison on this feature set rather than an assumption about which family would win.

OUTCOMES

Result. Delivered price transparency the business did not previously have — an analytical view of what drives trip cost, plus automated price estimation for arbitrary shipment configurations.

Sa3ar - Used-Car Fair-Price Estimation

CONTEXT

Situation. Used-car pricing in the local market runs on asking prices and intuition. Neither buyer nor seller has a reference point for what a specific car is actually worth, and no usable dataset existed to build one from.

Task. Assemble the data, then estimate a fair price for a used car from its attributes and condition.

Challenge. The problem started upstream of modelling: the market data existed only as unstructured classified listings, and asking price is a noisy proxy for value.

SOLUTION

Action. I scraped OLX listings to gather car specifications, mileage, tyre condition, accident history and asking price, then built a model to estimate a fair price from those characteristics.

Rationale. Acquiring the dataset was the differentiating part of the work. Once listings across the market are structured, the price surface is learnable — and no one else in the market had that view.

OUTCOMES

Result. A fair-price estimator built on a market dataset assembled from scratch, giving a defensible valuation for a used car from its specifications and condition.

AUC - Course Registration Demand Forecasting (PoC)

CONTEXT

Situation. A university has to decide, months in advance, how many seats to open in each course for the coming semester. Open too few and students cannot progress; open too many and staff and rooms are committed to empty classrooms. The planning was being done from last year's numbers and departmental judgement.

Task. Predict, as accurately as possible, the number of seats each course will need in an upcoming semester.

Challenge. Course demand is not a smooth time series — it is the aggregate of thousands of individual registration decisions that depend on what each student has already completed, what their programme requires next, and where they are in their degree. A course's history alone cannot see a cohort moving through a prerequisite chain toward it.

SOLUTION

Action. I built the forecast bottom-up rather than as a per-course trend, in two stages.

First, **a course-dependency graph mined from registration history**: association-rule mining over students' completed-course sets, producing rules of the form "students who have taken these courses go on to take that one", each with its own confidence. This is a data-derived progression map — it captures both the formal prerequisite structure and the informal paths students actually take, which the calendar does not record.

Second, **a per-student prediction model**. For every student, the rule base proposes the courses they are plausibly next to take; a gradient-boosted classifier (XGBoost) then predicts, from the student's own profile — school, major, department, gender, cumulative hours earned, GPA, whether they entered via a secondary school code, the level of the course, the season and year — which of those proposed courses they will actually register for. Summing the accepted predictions per course gives the seat forecast.

Validation was a genuine backtest: hold out an entire semester, mine rules and train only on what preceded it, then predict that semester cold. Accuracy was measured two ways — mean absolute error on the seat counts themselves, and, per course, how many of the students who actually enrolled had been individually predicted to. Reported errors on held-out semesters ran from under 1% on some large courses to the low tens of percent on others. I also built a direct per-course seat-count regression as a comparison baseline, and packaged the results as an interactive presentation showing predicted against actual enrolment per course and semester.

Rationale. Predicting individual students and summing them is what lets the forecast anticipate a bulge before it arrives — a large cohort clearing a prerequisite this year shows up in next year's seat count, which a per-course trend model cannot see until it has already happened. Scoring at the student level as well as the aggregate matters because two forecasts can produce the same total from completely different people, and only one of them supports timetabling.

OUTCOMES

Result. A proof of concept forecasting per-course seat requirements for an upcoming semester from a mined course-dependency graph plus per-student registration prediction, validated by holding out whole semesters and reported against actual enrolment course by course.

Toyota - Customer Data Quality and Deduplication

CONTEXT

Situation. Toyota's historical record of customer service visits was unreliable. Names carried typos, telephone numbers were implausible or invented, and the same customer appeared as several distinct records — so anything built on this data, from service reminders to lifetime-value analysis, was built on sand.

Task. Cleanse the historical appointment records and identify which visits actually belonged to the same customer.

Challenge. No single field resolved identity. VINs, names, phone numbers and tax numbers were each individually unreliable, and the truth had to be reconstructed from their agreement.

SOLUTION

Action. I cleansed the historical maintenance-appointment data — resolving name typos, removing illogical telephone numbers and similar contamination — then grouped likely duplicate customers by matching across vehicle identification numbers, customer names, telephone numbers and tax numbers together.

Rationale. Entity resolution on any one weak signal produces either missed matches or wrong merges. Requiring corroboration across several independent identifiers is what makes the result trustworthy enough to act on — and a wrong merge in customer data is far more expensive to unwind than a missed one.

OUTCOMES

Result. A cleansed, deduplicated customer history — the prerequisite for any downstream customer analytics, retention modelling or service communication that Toyota wanted to build.

Cargolinx - Freight Commodity Name Standardisation (PoC)

CONTEXT

Situation. A freight and logistics platform's shipment records describe their cargo in free text. The same commodity arrives spelled a dozen ways — singular and plural, misspelled, abbreviated, qualified — so any attempt to report on what was actually being moved, price by commodity type, or apply handling rules for perishables ran into a catalogue that had no canonical entries.

Task. Give the business a way to collapse free-text commodity descriptions onto a standard set of commodity names, and to place those names within a category hierarchy.

Challenge. Standardisation cannot be a one-off cleanup, because new descriptions keep arriving. It needed to be an interactive tool the operations team could query — and it had to group variants transitively, since "strawberry", "strawberries" and "strawbery" must all land together even when no single pair is the obvious anchor.

SOLUTION

Action. I built an access-controlled R Shiny tool that, given any query term, retrieves the commodity names most similar to it from the observed catalogue using **Jaro-Winkler string distance under a tuned similarity threshold**, and groups them **transitively**: each match is used as a new query and the process repeats until the group stops growing, so a chain of near-misses resolves into one cluster rather than several. Matches are returned alongside how often each variant actually occurs in the data, so the operator standardising a group can see which spelling is the dominant one rather than guessing. The tool sits on top of a commodity taxonomy mapping each item to its parent category, including the distinctions that matter operationally — perishable versus non-perishable produce, for instance, which drives handling and routing.

Rationale. Fully automatic deduplication of commodity names is the wrong target: a wrong merge in a shipment catalogue silently corrupts every downstream report and is expensive to unwind. Making the tool a fast, interactive assistant — propose the cluster, show the evidence, let a person confirm — puts the judgement where it belongs while removing all of the searching. Weighting on prefix agreement, which is what Jaro-Winkler does, suits commodity and brand names specifically, since they tend to diverge at the end rather than the beginning.

OUTCOMES

Result. A proof-of-concept standardisation tool that turns a free-text commodity field into reviewable candidate groups with occurrence counts, mapped onto a parent-category hierarchy — the prerequisite for commodity-level pricing, reporting and handling rules.

Abou Ghaly Motors - Service-Visit Analytics (PoC)

CONTEXT

Situation. An automotive dealer group's after-sales business depends on knowing how its customers' vehicles are actually used — how fast they accumulate mileage determines when the next service is due and what work it should include. That information exists only as odometer readings captured at service visits, and those readings are entered by hand.

Task. Reconstruct per-vehicle usage from service history, and identify the readings and visits that cannot be trusted.

Challenge. Hand-entered odometer values contain transpositions, unit confusion and outright impossible numbers, and a single bad reading corrupts the usage rate on both sides of it. The same vehicle can also appear more than once on one visit date across different source files, with different readings.

SOLUTION

Action. Working from a sample of maintenance records covering **more than 1,100 vehicles** across three source systems, I joined the visit, notification and maintenance-package records per vehicle, reconciled multiple readings for the same vehicle-day down to one value, and derived each vehicle's **mileage accumulation rate between consecutive visits**. Each interval's rate is then expressed as a deviation from that vehicle's own median rate, and flagged using an **interquartile-range fence** — visits whose implied usage falls far outside the vehicle's normal band are marked as suspect ("RED" visits). I aggregated those flags by order type and service centre to show where implausible readings concentrate, and delivered the whole thing as an interactive dashboard with per-vehicle rate tables colour-coded by deviation, alongside the breakdown of flag rates per order type.

Rationale. Comparing each vehicle against its own median rather than a fleet-wide average is the key choice: a taxi and a weekend car have legitimately different usage, and a global threshold would flag one and miss the other. An IQR fence rather than a fixed cut-off keeps the rule from needing to be re-tuned per fleet. Reporting the flag rate by order type and service centre turns a data-quality exercise into an operational finding — implausible readings clustering at one centre or one order type is a process problem, not a random error.

OUTCOMES

Result. A proof of concept converting hand-entered service records into per-vehicle usage rates with a defensible anomaly flag, plus a view of where in the service network unreliable data originates — the prerequisite for any mileage-driven maintenance scheduling or service-package recommendation.

RESEARCH & OPEN SOURCE

dbparser - Open-Source Drug-Database Integration Package

CONTEXT

Situation. Drug-discovery research depends on public databases like DrugBank and KEGG, but every research group was independently writing throwaway code to parse them — solving the same integration problem repeatedly and inconsistently.

Task. Supervise development of an open-source R package that parses and integrates these heterogeneous sources, and set the research direction and feature priorities behind it.

Challenge. The sources differ in schema and format, and the package had to serve genuine drug-discovery workflows rather than merely reading files — while meeting the software-quality bar that open-source adoption demands.

SOLUTION

Action. I co-developed and supervised dbparser, an R package for parsing public drug databases including DrugBank and KEGG. I defined the software requirements and research directions in line with drug-discovery needs, prioritised features, and guided design decisions and comprehensive testing. Parsed heterogeneous sources are structured into a unified R object format — the DrugVerse object — so downstream analysis works against one consistent representation regardless of source.

Rationale. Unifying the output representation is what turns a parser into infrastructure: researchers write analysis once and it works across sources. Centralising recurring integration work into a tested package is the difference between solving the problem once and solving it in every lab separately.

OUTCOMES

Result. 50K+ downloads and active community adoption — reusable research infrastructure now relied on well beyond the team that built it.

PhD Research - Drug-Target Interaction Prediction

CONTEXT

Situation. Determining whether a drug binds a given protein target experimentally is slow and expensive, so computational prediction is used to prioritise which candidates are worth testing at the bench. The prediction problem is hard for structural reasons: known interactions are sparse, and confirmed positives are vastly outnumbered by unknowns.

Task. Conduct predictive-ML research on drug-target interactions and publish it in peer-reviewed venues.

Challenge. Severe class imbalance, sparse observations, and the need to integrate chemical and biological representations of very different kinds — all under evaluation rigorous enough to survive peer review.

SOLUTION

Action. I developed and evaluated methods spanning ensemble learning, matrix factorisation, manifold learning, dimensionality reduction and class-imbalance-aware learning, including graph-regularised matrix factorisation. The work spanned MATLAB, R, Python, Java and MySQL.

Rationale. These families attack the problem from complementary angles — matrix factorisation recovers latent interaction structure, manifold learning and dimensionality reduction address representation geometry, and imbalance-aware ensembles handle minority-class detection. No single approach addresses all three, which is why the research programme covered them in combination.

OUTCOMES

Result. 4 peer-reviewed papers and 1 book chapter, presented at international conferences, including a **Best Paper Award at InCoB 2016 (BMC Bioinformatics)**. Published in *Briefings in Bioinformatics*, *IEEE/ACM TCBB*, *Methods (Elsevier)* and *BMC Bioinformatics*.

EARLIER ENGINEERING

Earlier Software Projects - BRS and OTMS

CONTEXT

Situation. At United OFOQ I worked on two enterprise systems with very different demands. The Baggage Reconciliation System (BRS, 2008-2010) handles baggage in airport sorting areas, where a mis-sorted bag is a failure a passenger experiences directly. The OFOQ Talent Management System (OTMS, 2010-2011) is a configurable web-based HR platform serving clients with materially different HR processes.

Task. Deliver updates, resolve reported issues, support deployments, and improve product functionality and documentation across both.

Challenge. BRS was operated largely through handheld barcode scanners in a live sorting area — a constrained interface used under time pressure, where usability is an operational requirement rather than a nicety. OTMS had to support client-defined HR processes across payroll, recruitment, health insurance, resource planning and training, with multilingual support and a security model strong enough for confidential employee data.

SOLUTION

Action. On BRS, I added critical updates and new functionality, fixed issues raised internally and by customers, provided direct customer support, improved the visual and operational usability of the scanner-facing interface, updated and extended the documentation for new components, and travelled to Amman, Jordan to carry out on-site deployment.

On OTMS, I worked within the development team, built the reporting functionality using JasperReports, and contributed to data dictionaries, user-defined codes and multilingual support. The stack across both spanned Java EE, JSF/JSP, JavaScript, EJB 3.0, JPA, IBM WebSphere, ActiveVOS, Glassfish, Liferay, IBM DB2 and Oracle.

Rationale. The scanner usability work mattered more than it appears: a UI that costs a handler an extra two seconds per bag is an operational cost multiplied by every bag through the airport. On OTMS, data dictionaries and user-defined codes are the mechanism through which one codebase adapts to each client's HR vocabulary without a fork.

OUTCOMES

Result. Three years of production enterprise software delivery across two live systems — including customer-facing support, documentation ownership and international on-site deployment — the engineering foundation underneath the production ML work that followed.